

SYSC 3020: Introduction to Software Engineering

Assignment 1: Requirements – Lab Walkthrough

Rinkesh Joshi · rinkeshjoshi@cmail.carleton.ca

SYSC 3020: Introduction to Software Engineering

Assignment 1: Requirements – Lab Walkthrough

Fall 2026

TA: Rinkesh Joshi – rinkeshjoshi@cmail.carleton.ca

Lab: LE1 · Wednesday 2:35 PM – 5:25 PM

Room: Mackenzie Building 4233

Assignment due: Monday, October 05, 2026 by 11:59 PM

The Three Tasks at a Glance

Task A

Map the Core

Read the JPacman source code and document it.

- One table per package (`board`, `level`, `npc`, `points`)
- Columns: Class · File path · Responsibility · Key methods
- A numbered Core Events list

Output: "Code Map" appendix in the SRS

Task B

Write the SRS

Create an SRS in Confluence with Requirement Yogi keys.

- B1: Introduction & scope
- B2: Actors
- B3: Functional reqs (≥ 12)
- B4: User stories (≥ 8)
- B5: NFRs (≥ 8)
- B6: Assumptions

Output: SRS.pdf (exported from Confluence)

Task C

Review for Defects

Check your own requirements for quality problems.

- Manual checklist (ISO/IEC/IEEE 29148)
- Build an LLM reviewer (LangChain + Ollama)
- Judge every LLM finding: accept or reject + reason

Output: `llm_req_review.py`

- Defect Review section in SRS

Plus a Sprint process layer running across all tasks in Jira (stand-ups, board, retrospective).

What You Submit

Deliverables (on Brightspace)

1. SRS.pdf — exported from Confluence
 - Sections B1-B6
 - Code Map appendix (Task A)
 - Defect Review section (Task C)
 - Team & Process section
2. llm_req_review.py — your completed LangChain reviewer script
3. Jira board link — shows sprint activity

Penalties to Watch

- Missing SRS.pdf → -2 marks
- Missing llm_req_review.py → -2 marks
- Late → -10% per day (max 48 hrs)
- Raw LLM output without judgment → 0 on Quality criterion
- NOT submitted separately:
`requirements.csv`, `llm-review-output.md`
(content goes inside SRS.pdf)

Grading: 4 criteria × 4 pts = 16 total — Completeness, Grounding, Quality, Process

Grading Rubric – Quick View

Completeness & Coverage (4 pts)

All elements present, counts met, SRS covers in-scope logic thoroughly.

Grounding & Traceability (4 pts)

Every functional req traces to a verified class + method; Task A map correct.

Quality & Defect Review (4 pts)

Requirements unambiguous, atomic, verifiable; thorough defect review with documented fixes.

Process & Reporting (4 pts)

Jira board current with real sprint activity; AI usage disclosed; SRS correctly formatted.

Score Range	Meaning
14-16	Excellent work
10-13	Good work
6-9	Some improvement needed
0-5	Much improvement needed

Setup › Install JDK 11

This project requires JDK 11 – newer versions may cause build failures.

Download: <https://www.oracle.com/ca-en/java/technologies/javase/jdk11-archive-downloads.html>

(Oracle account required – free to create)

Windows

- Download the Windows x64 Installer (.exe)
- Run the installer, accept defaults
- The installer sets `JAVA_HOME` automatically
- If not, set it manually:

```
# PowerShell (run as admin)
[System.Environment]::SetEnvironmentVariable(
    "JAVA_HOME",
    "C:\Program Files\Java\jdk-11",
    "Machine"
)
```

macOS

- Download the macOS DMG Installer (ARM64 for Apple Silicon, x64 for Intel)
- Open the .dmg and run the .pkg installer
- Or use Homebrew:

```
# Homebrew (recommended)
brew install openjdk@11
```

- Then link it:

```
sudo ln -sf \
    $(brew --prefix
    openjdk@11)/libexec/openjdk.jdk \
    /Library/Java/JavaVirtualMachines/openjdk-
    11.jdk
```

Setup › Verify Java

Windows

```
# Check if Java is installed
java -version

# Check the compiler
javac -version

# Expected output:
# java version "11.0.x" ...
# javac 11.0.x
```

macOS

```
# Check if Java is installed
java -version

# Check the compiler
javac -version

# Expected output:
# openjdk version "11.0.x" ...
# javac 11.0.x
```

If you see a different major version (e.g., 17, 21), you need to switch → next slide.

Setup › Switch Java Versions

Only needed if `java -version` shows something other than 11.

Windows

```
# PowerShell (run as admin)
[System.Environment]::SetEnvironmentVariable(
    "JAVA_HOME",
    "C:\Program Files\Java\jdk-11",
    "Machine"
)

# Restart your terminal after this!
```

Check with: `echo %JAVA_HOME%` (cmd) or
`$env:JAVA_HOME` (PowerShell)

macOS

```
# List all installed JDKs
/usr/libexec/java_home -V

# Switch to JDK 11 for this session
export JAVA_HOME=$( \
    /usr/libexec/java_home -v 11)

# Make it permanent — add to ~/.zshrc
echo 'export JAVA_HOME=$( \
    /usr/libexec/java_home -v 11)' \
    >> ~/.zshrc
```

Setup › Clone the Repo & Build

Windows

```
# Option 1: Clone with Git
git clone
https://github.com/SYSC3020-F2026/sysc-termproject-starter.git

cd sysc-termproject-starter

# Option 2: Download ZIP from GitHub
#   Code → Download ZIP → extract
```

Build

```
# Use the Gradle wrapper
./gradlew build
```

You should see BUILD SUCCESSFUL.

macOS

```
# Option 1: Clone with Git
git clone
https://github.com/SYSC3020-F2026/sysc-termproject-starter.git

cd sysc-termproject-starter

# Option 2: Download ZIP from GitHub
#   Code → Download ZIP → extract
```

Build

```
# Make gradlew executable (once)
chmod +x gradlew

# Use the Gradle wrapper
./gradlew build
```

You should see BUILD SUCCESSFUL.

Setup > What the Repo Looks Like

Key folders

Folder	What's inside
<code>src/</code>	JPacman source – read this
<code>doc/</code>	<code>scenarios.md</code> – behaviour docs
<code>scripts/</code>	<code>llm_req_review.py</code> (Task C)
<code>build/</code>	Compiled output (after build)
<code>gradle/</code>	Wrapper files (don't touch)

Key files

File	Purpose
<code>AI_USAGE.md</code>	Document AI tool usage
<code>TEAM_CHARTER.md</code>	Team charter template
<code>build.gradle</code>	Build config (don't modify)
<code>gradlew</code>	Gradle wrapper (Mac/Linux)
<code>gradlew.bat</code>	Gradle wrapper (Windows)

Setup › The Source Tree You Care About

```
src/main/java/nl/tudelft/jpacman/
├── board/           ← Task A scope
│   ├── Board.java
│   ├── BoardFactory.java
│   ├── Direction.java
│   ├── Square.java
│   └── Unit.java
├── level/          ← Task A scope
│   ├── Level.java
│   ├── MapParser.java
│   ├── PlayerCollisions.java
│   └── ...
├── npc/            ← Task A scope
│   └── ghost/
│       ├── Blinky / Pinky / Inky / Clyde
│       ├── Navigation.java
│       └── GhostFactory.java
├── points/         ← Task A scope
│   ├── PointCalculator.java
│   └── ...
```

These four packages are your entire scope for the assignment. Everything else is out of scope.

Task A > What You Actually Do

Step 1: Read the code in the four packages

Open `src/main/java/nl/tudelft/jpacman/` and explore: `board` · `level` · `npc` (+ `npc/ghost`) · `points`

Also read `doc/scenarios.md` – the game's own behaviour documentation.

Step 2: Build a table for each package

For every class, record these four columns:

Class	File path	Responsibility	Key methods
Navigation	<code>.../npc/ghost/</code>	Path-finding	<code>shortestPath()</code>
Board	<code>.../board/</code>	Grid structure	<code>withinBorders()</code>
...etc...			

(Full paths like `src/main/java/nl/tudelft/jpacman/npc/ghost/Navigation.java`)

Task A > Core Events & Hint

Step 3: Write the Core Events list

Numbered list of game behaviours tied to the code:

1. Player moves onto a pellet → score increases, pellet removed (
 `PlayerCollisions.playerVersusPellet` → `PointCalculator`)
2. ...discover more by reading the code...

This entire output becomes the "Code Map" appendix in your SRS on Confluence.

 Hint: Start with `doc/scenarios.md` for the obvious events, then dig into `PlayerCollisions` and the ghost classes for what the docs don't tell you.

Task B › What Are These Tools?

Jira

Work tracking board. You create work items (tasks), assign them to people, and move them through columns: To Do → In Progress → Done.

Used here to run your sprint: planning, stand-ups, retrospective.

Note: Jira now calls "projects" spaces and "issues" work items.

Confluence

Team wiki / document editor. You create pages with rich text, tables, headings – like Google Docs but inside Atlassian.

Used here to write your SRS and export it to PDF.

Requirement Yogi

Confluence add-on for requirements management. Type `/requirement` on a Confluence page and it assigns a tracked key (e.g., SJC-001).

Used here to tag each functional and non-functional requirement with a key.

Task B > Step 1 – Create Your Atlassian Workspace

Sign up (one person per team, then invite the other two):

<https://www.atlassian.com/software/jira/free>

Important

- Choose the Free plan – it supports up to 10 users
- Do NOT enter credit card or payment details – the free tier is all you need for a team of 3
- Each team member must create their own Atlassian account and join the workspace – the board history needs to show who did what

After signup you get:

- Jira – for your sprint board (To Do → In Progress → Done)
- Confluence – for writing your SRS page
- Both are at <https://yourteamname.atlassian.net>

Task B > Invite Your Team & Assign Roles

Invite members to the Jira space

1. In the left sidebar → next to your space name → click More actions (•••)
2. Click Add people
3. Enter each teammate's name or email
4. Set their role to Member (or Administrator so everyone can manage the board)
5. Click Add — they'll get an email invite

Each person must accept the invite

- Teammates click the link in the invite email
- They create their own Atlassian account (if they don't have one)
- Once accepted, their name shows up on work items and board history

Why this matters for grading

The rubric checks that the Jira board shows who did what — if everyone uses one account, there's no individual history. Each member needs their own account so that task assignments, status changes, and stand-up comments are all attributed correctly.

Task B › Step 2a – Create a Jira Space

Create a Scrum space

1. In the left sidebar → hover over Spaces → click Create space (+)
2. Select the Scrum template → Use template
3. Name it something like "SYSC3020-A1"
4. Choose Team-managed (simpler for small teams)

Review the configuration before creating

- Work types: keep Epic, Task, Story (remove Bug if you want)
- Statuses: To Do · In Progress · Done (matches the assignment)
- Views: Board + Backlog (both enabled by default)
- Click Create

The three roles (assign now, rotate later)

- Product Owner – owns the requirements
- Scrum Master – runs stand-ups, removes blockers
- QA Lead – reviews quality of deliverables

Task B > Step 2b – Sprint Planning in Jira

Create the Epic

- Click Create (top navigation bar) → set work type to Epic
- Name it: "A1 – Requirements" → click Create
- Or: go to Backlog (left sidebar) → open Epic panel via View Settings → + Create epic

Add work items (tasks) to the epic

- Click Create (top nav) → set work type to Task → fill in summary → select your Epic under "Epic Link" → Create
- Or: in the Backlog, drag-and-drop existing work items into the Epic panel
- Assign each work item to a team member

Create and start a sprint

1. In the Backlog view (left sidebar under your space)
2. Click Create Sprint – a sprint section appears
3. Drag work items from the Backlog into the sprint
4. Click Start Sprint → set name + dates → Start

Stand-ups: ≥ 2×/week as Jira comments on work items (done / doing / blocked).

Guide: <https://www.atlassian.com/agile/tutorials/how-to-do-scrum-with-jira>

Task B > Step 3 – Install Requirement Yogi

Add the app to Confluence

1. In Confluence → Apps (top nav) → Explore more apps
2. Search for "Requirement Yogi"
3. Click Try it free → select your site → Start free trial

How to use it on a page

- While editing a page, type `/requirement`
- Yogi inserts a requirement macro with a tracked key (e.g., SJC-001)
- Each functional req and NFR needs a Yogi key
- User stories and assumptions do NOT need keys

Tutorials

- Marketplace: <https://marketplace.atlassian.com/apps/1212523>
- Video tutorials: search "Requirement Yogi Cloud tutorial" on YouTube
- Official docs: <https://docs.requirementyogi.com/cloud/requirement-types>

Task B › Step 4a – Create the SRS Page

 In Confluence → Create → Blank page

Title it: "SRS – JPacman Core"

Then add these six section headings (no template – the assignment IS the template):

 B1: Introduction & scope

One paragraph – name the game, state what is in and out of scope.

 B2: Actors

External actors + internal collaborators the core logic depends on.

 B3: Functional requirements (≥ 12, ≥ 3 code-only)

"The system shall..." – cite class + method. Use `/requirement` for each.

Task B > Step 4b – SRS Sections (continued)

B4: User stories (≥ 8)

"As a... I want... so that..." with Given/When/Then acceptance criteria.

B5: Non-functional requirements (≥ 8)

Measurable, tagged: maintainability / testability / portability / usability. Use `/requirement` for each.

B6: Assumptions

Numbered list.

Remember: Yogi keys

- Functional requirements (B3) → need `/requirement` keys (e.g., SJC-001)
- Non-functional requirements (B5) → need `/requirement` keys
- User stories (B4) and Assumptions (B6) → do NOT need keys

Task B > What a Good Requirement Looks Like

Functional requirement format

```
SJC-001: Pellet consumption scores points.  
When the player moves onto a square containing a pellet,  
the system shall add the pellet value to the score and remove  
the pellet.  
(Source: PlayerCollisions.playerVersusPellet,  
PointCalculator.consumedAPellet.)
```

Key parts: Yogi key · descriptive title · "the system shall..." · class + method citation.

User story format

```
As a player,  
I want the score to increase when I eat a pellet,  
so that I can track my progress.  
  
Given a board with pellets,  
When the player moves onto a pellet square,  
Then the pellet is removed and the score increases by the pellet value.
```

Code-only requirements

Behaviours not documented in `scenarios.md` — found only by reading `src/`. You need ≥ 3 of these. (The border-tunnel example in the assignment doesn't count.)

Task B > Step 5 – Export & Extras

Exporting the SRS

In Confluence: page → ⋯ (three dots) → Export → PDF

This produces SRS.pdf – your main deliverable.

Also create requirements.csv

A simple CSV file for the LLM reviewer in Task C:

```
Key,Description
SJC-001,"When the player moves onto a square containing a pellet, the system shall..."
SJC-002,"When a ghost collides with the player, the system shall..."
```

One row per requirement. You do not submit this file separately – it's just input for `llm_req_review.py`.

Don't forget these sections in the SRS

- Code Map appendix (from Task A)
- Defect Review section (from Task C – coming next)
- Team & Process section: roles, ways of working, retrospective (keep/change/try), AI Usage note

Task C > What You're Building

An LLM-powered requirements reviewer

You build a small Python script that:

1. Reads your `requirements.csv`
2. Sends each requirement to a local LLM (Ollama + llama3.2)
3. Checks for ISO/IEC/IEEE 29148 quality defects
4. Outputs candidate defects

Then you judge every finding – accept or reject with a reason.

Two parts to the defect review

- Manual checklist (Step 1) – you score requirements yourself against ISO 29148
- LLM reviewer (Steps 2-5) – the script proposes, you decide

Both go into the Defect Review section of your SRS.

 Submitting raw LLM output without judgment → 0 on Quality criterion

Task C › Step 1 – Manual Checklist Review

Score each requirement against ISO/IEC/IEEE 29148

The seven characteristics to check:

- Unambiguous – only one interpretation possible
- Complete – all conditions and responses described
- Consistent – no contradictions with other requirements
- Verifiable – can be tested with a concrete test
- Singular – describes exactly one thing (no "and/or")
- Feasible – can be implemented within scope
- Traceable – links to a specific source (class + method)

Red flags to look for

- Weak words: "fast", "user-friendly", "should", "easy"
- Missing actor: who triggers the behaviour?
- Non-atomic: "and/or" in a single requirement
- Stories without Given/When/Then

Fix what you find before running the LLM reviewer.

Task C > Step 2 – Install Ollama (Local LLM)

Windows

```
# Install Ollama
winget install Ollama.Ollama

# Pull the model (~2 GB, once)
ollama pull llama3.2
```

After install, Ollama runs as a background service automatically.

Verify it's working

```
ollama list
# Should show llama3.2
```

macOS

```
# Install Ollama
brew install ollama

# Start the background server
brew services start ollama

# Pull the model (~2 GB, once)
ollama pull llama3.2
```

Verify it's working

```
ollama list
# Should show llama3.2
```

No API key needed – everything runs locally on your machine.

Task C › Step 3 – Python Environment Setup

Windows

```
# Create a virtual environment
py -m venv .venv

# Install LangChain libraries
.venv\Scripts\pip install \
    langchain langchain-ollama
```

macOS

```
# Create a virtual environment
python3 -m venv .venv

# Install LangChain libraries
.venv/bin/pip install \
    langchain langchain-ollama
```

Requires Python 3.10+ – check with `python3 --version` or `py --version`.

Task C > Step 4 – Complete the Script

Open `scripts/llm_req_review.py`

You fill in two TODOs – that's the graded part:

TODO 1: The Prompt

Define each ISO 29148 characteristic as a concrete defect test:

```
# Not just "unambiguous" – tell the model WHAT to look for:
# - Unambiguous: flag if a requirement uses vague terms
#   like "fast", "appropriate", "user-friendly", or
#   can be read in more than one way.
# - Also: ask for cross-requirement contradictions
```

TODO 2: The Model Call

```
from langchain_ollama import ChatOllama

model = ChatOllama(
    model="llama3.2",
    temperature=0      # reproducible output
)
print(model.invoke(prompt).content)
```

Task C › Step 5 – Run & Judge

Run the reviewer

Windows

```
.venv\Scripts\python \  
scripts/llm_req_review.py \  
requirements.csv \  
> llm-review-output.md
```

macOS

```
.venv/bin/python \  
scripts/llm_req_review.py \  
requirements.csv \  
> llm-review-output.md
```

Judge EVERY finding

For each line the LLM outputs:

- Accept – yes, this is a real defect → write a one-line reason + fix it
- Reject – no, the LLM is wrong → write a one-line reason why

Put it ALL in the Defect Review section of your SRS

- Your manual checklist findings
- The raw LLM reviewer output
- Your prompt (from the script)
- Your accept/reject decisions with reasons
- Confirmed defects and their fixes

